

Chapter Four

Software Development Methodologies and Paradigms

Contents

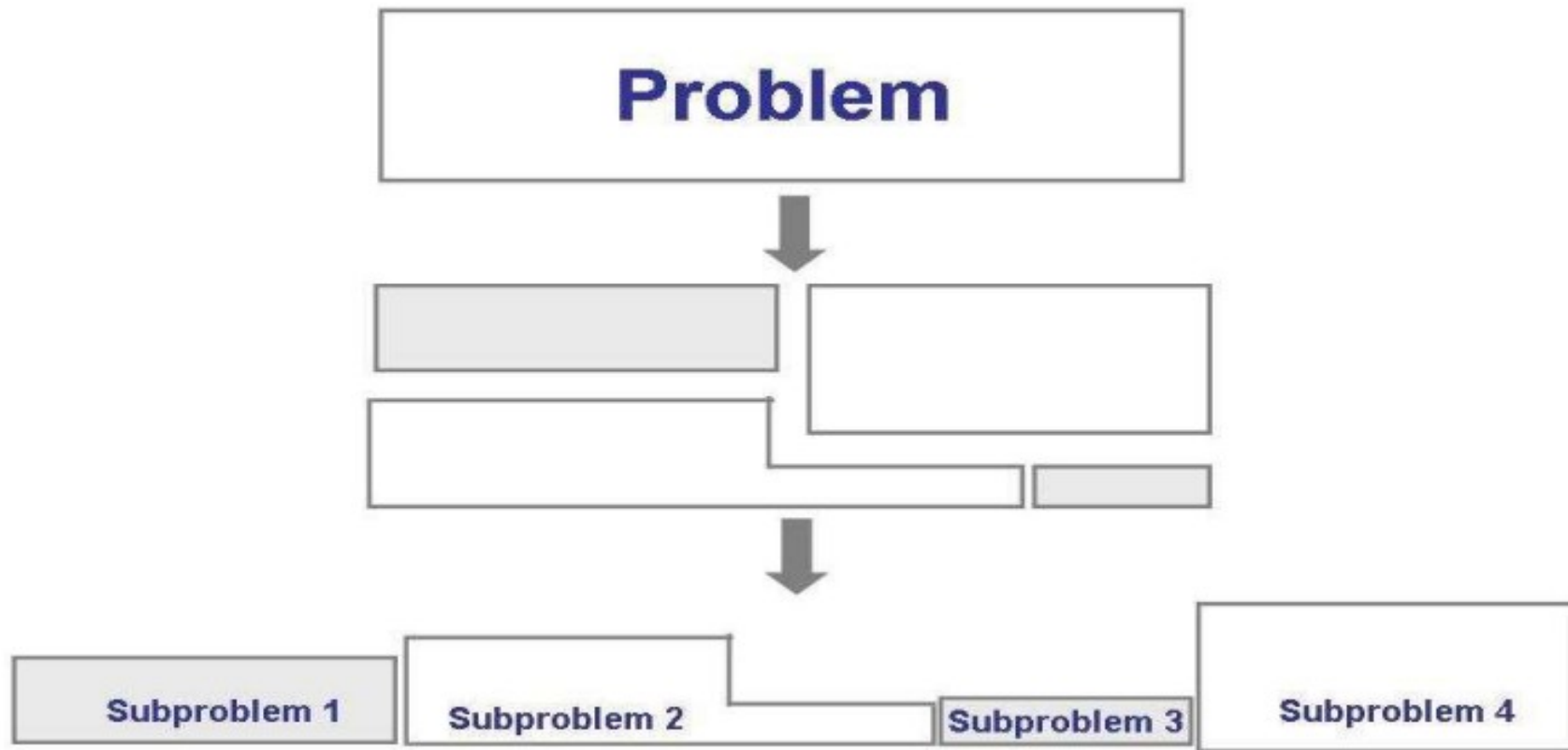
- Software Engineering Practice
- Essence of SE Practice
- Core Principles of Software Engineering
- Types of Practice
- Software Engineering Methodologies

Recall

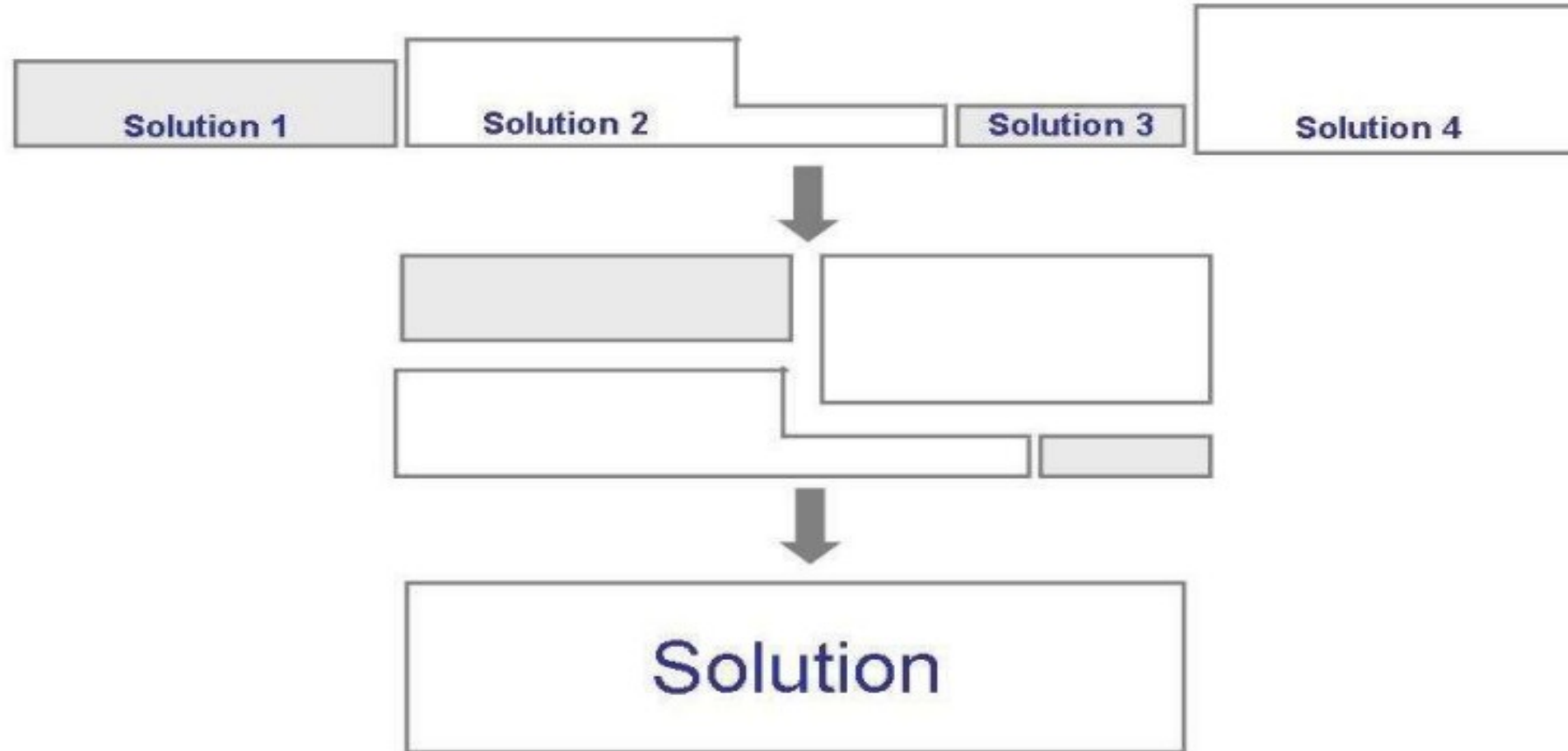
- Software Development requires **analysis** and **synthesis**
 - **Analysis:** **decompose** a large problem into smaller, understandable pieces
 - **Synthesis:** **build (compose)** software from smaller building blocks

Software Engineering: Solving Problem

- The analysis process



- The synthesis process



Software Engineering Practice

- Consists a collection of **concepts, principles, methods, and tools** that a software engineer calls upon on a daily basis.
- Equips managers to **manage** software projects and software engineers to **build** computer programs
- Provides necessary **technical** and **management** knowledge how to **produce** a software product.

Essence of SE Practice

1. Understand the problem (communication and analysis)
 - Who has a stake in the solution to the problem?
 - What are the unknowns? and what (data, function, behavior) are required to properly solve the Problem?
 - Is it possible to represent smaller problems that may be easier to understand.
 - Can the problem be represented graphically? Can analysis model be created?

2. Plan a solution (planning, modeling and software design)

- Have you seen similar problems like this before?
- Has a similar problem been solved? If so, are the elements of the solution reusable?
- Can sub problems be defined and are solutions available for the sub problems?

3. Carry out the plan (construction; code generation)

- Does the **solution** conform to the **plan**? Is the source code traceable back to the **design**?
- Is each component of the solution **correct**? Has the design and code been reviewed?

4. Examine the results for accuracy (testing and quality assurance)

- Is it possible to **test each component** of the solution? Has a reasonable testing strategy been implemented?
- Does the **solution** produce **results** that conform to the data, function, and behavior that are required?
- Has the Software been **validated** against all stakeholders requirement?

Core Principles of Software Engineering

1. Remember the reason that the software exists

- The software should **provide value** to its users and satisfy the requirements

2. Keep it **simple**

- All design and implementation should be **as simple as possible**

3. Maintain the **vision** of the project

- A clear vision is essential to the **project's success**

4. Others will **consume** what you produce

- Always specify, design, and implement knowing that someone else **will later** have to **understand** and **modify** what you did

5. Be open to the future

- Never design yourself into a corner; build software that can be easily **changed** and **adapted**.

6. Plan ahead for software reuse

- Reuse of software **reduces the long-term cost** and **increases the value of the program** and the reusable components.

7. Think, then act

- Placing clear, complete thought **before action** will almost always produce **better results**

Types of Practices(SDLC)

- Communication Practice
- Planning Practice
- Modeling Practice
- Construction Practice
- Testing Practice
- Deployment Practice

Communication Practice

1. Listen to the speaker and concentrate on what is being said
2. Prepare before you meet by researching and understanding the problem
3. Someone should facilitate the meeting and have an agenda

4. Face-to-face communication is best, but also have a document or presentation to focus the discussion
5. Take notes and document decisions
6. Strive for collaboration and consensus
7. Stay focused on a topic; modularize your discussion

8. If something is unclear, draw a picture

9. Move on to the next topic

a) after you agree to something,

b) if you cannot agree to something, or

c) if a feature or function is unclear and cannot be clarified at the moment

10. Negotiation is not a contest or a game; it works best when both parties win

Planning Practice

1. Understand the scope of the project
2. Involve the customer in the planning activity
3. Recognize that planning is iterative; things will change
4. Estimate based on only what you know
5. Consider risk as you define the plan

6. Be realistic on how much can be done each day by each person and how well

7. Adjust **granularity** as you define the plan

8. Define how you intend to **ensure quality**

9. Describe how you intend to **accommodate change**

10. Track the plan frequently and make adjustments as required

Modeling Practice (Analysis)

1. The **information domain** of a problem (the data that flows in and out of a system) must be represented and understood
2. The **functions** that the software performs must be defined
3. The **behavior of the software** (as a consequence of external events) must be represented

- 4. The models that depict information, function, and behavior must be partitioned in a manner that uncovers detail in a layered (or hierarchical) fashion
- 4. The **analysis task** should move from essential information toward implementation detail

Modeling Practice (Design)

1. The design should be traceable to the analysis model
2. Always consider the software architecture of the system to be built
3. Design of data is as important as design of processing functions

4. Interfaces (both internal and external) must be designed with care

5. User interface design should be tuned to the needs of the end-user and should stress ease of use

6. Component-level design should be functionally independent (high cohesion)

7. Components should be loosely coupled to one another and to the external environment

8. Design representations (models) should be easily understandable

9. The design should be developed iteratively; with each iteration, the designer should strive for greater simplicity

Construction Practices (Before Coding)

1. Understand the problem you are trying to solve
2. Understand basic design principles and concepts
3. Pick a programming language that meets the needs of the software to be built and the environment in which it will operate
4. Select a programming environment that provides tools that will make your work easier
5. Create a set of unit tests that will be applied once the component you code is completed

Construction Practices (When Coding Begin)

1. Constrain your algorithms by following structured programming practices
2. Select data structures that will meet the needs of the design
3. Understand the software architecture and create interfaces that are consistent with it
4. Keep conditional logic as simple as possible

- 5. Create nested loops in a way that makes them easily testable
- 5. Select meaningful variable names and follow other local coding standards
- 5. Write code that is self-documenting
- 5. Create a visual layout (e.g., indentation and blank lines) that aids code understanding

Construction Practices (When Coding End)

1. Conduct a code walkthrough
2. Perform unit tests (black-box and white-box) and correct errors you have uncovered
3. Refactor the code

Testing Practice

1. All tests should be traceable to the software requirements
2. Tests should be planned long before testing begins
3. The Pareto principle applies to software testing
 - 80% of the uncovered errors are in 20% of the code

4. Testing should begin “in the small” and progress toward testing “in the large”

- Unit testing --> integration testing --> validation testing --> system testing

5. Exhaustive testing is not possible

Deployment Practice

1. Customer expectations for the software must be managed
 - Be careful not to promise too much or to mislead the user
2. A complete delivery package should be assembled and tested
2. A support system must be established before the software is delivered

- 4. Appropriate instructional materials must be provided to end users
- 4. Buggy software should be fixed first, delivered later

Software Engineering Methodologies(SDPM)

Software Engineering Methodologies

- SE methods provide an organized and systematic approach to developing software for a target computer.
- There are numerous methods from which to choose, and it is important for the software engineer to choose an appropriate method or methods.
- The choice can have a dramatic effect on the success of the software project.

Software Engineering Methodologies

- Some of the selected SE methods are: Heuristic Methods, Formal Methods, Prototyping Methods, and Agile Methods.

Heuristic Methods

- Are those experience-based SE methods that have been and are fairly widely practiced in the software industry.
- The Heuristic Methods has three discussion categories.

Software Engineering Methodologies (Cont.)

- i. Structured analysis and design methods
- ii. Data modeling methods
- iii. Object oriented analysis and design methods.

Structured analysis and design methods:

- ✓ The software model is developed primarily from a functional or behavioral viewpoint
- ✓ Starting from a high-level view of the software (including data and control elements) and then progressively decomposing or refining the model components through increasingly detailed designs
- ✓ The detailed design eventually converges to very specific details or specifications of the software that must be coded (by hand, automatically generated, or both), built, tested, and verified.

Data Modeling Methods:

- ❖ The data model is constructed from the viewpoint of the data or information used. Data tables and relationships define the data models.
- ❖ Used primarily for defining and analyzing data requirements supporting database designs or data repositories typically found in business software, where data is actively managed as a business systems resource or asset.

Software Engineering Methodologies (Cont.)

Object-Oriented Analysis and Design Methods:

- The object-oriented model is represented as a collection of objects that encapsulate data and relationships and interact with other objects through methods.
- Objects may be real-world items or virtual items

- The software model is constructed using diagrams to constitute selected views of the software.
- Progressive refinement of the software models leads to a detailed design.
- The detailed design is then either evolved through successive iteration or transformed (using some mechanism) into the implementation view of the model.

Software Engineering Methodologies (Cont.)

Prototyping Methods

- Software prototyping is an activity that generally creates **incomplete** or **minimally functional versions** of a software application,
- Usually the method used for trying out **specific new features**, **soliciting feedback** on **software requirements** or **user interfaces**, further exploring software requirements, **software design**, and other cases.

Software Engineering Methodologies (Cont.)

- The software engineer selects a prototyping method to understand the least understood aspects or components of the software first;
- The prototyping approach is in contrast with other software engineering methods that usually begin development with the most understood portions first.

- Prototyped product does not become the final software product without extensive development rework or refactoring.

Prototyping Style: This addresses the various approaches to develop prototypes.

- The style chosen is based on the type of results the project needs, the quality of the results needed, and the urgency of the results.

Software Engineering Methodologies (Cont.)

Prototyping Target: The target of the prototype activity is the specific product being served by the prototyping effort.

- Examples of prototyping targets include a requirements specification, an architectural design element or component, an algorithm etc.
- **Prototyping Evaluation Techniques:** A prototype may be used or evaluated in a number of ways by the software engineer or other project stakeholders.

Software Engineering Methodologies (Cont.)

- Primarily by the underlying reasons that led to prototype development in the first place.
- Prototypes may be evaluated or tested against the actual implemented software or against a target set of requirements (for example, a requirements prototype)
- The prototype may also serve as a model for a future software development effort (for example, as in a user interface specification).

Software Engineering Methodologies (Cont.)

Agile Methods

- Considered lightweight methods in that they are characterized by
 - Short,
 - Iterative development cycles,
 - Self-organizing teams,
 - Simpler designs,
 - Code refactoring,

Software Engineering Methodologies (Cont.)

- Test-driven development,
- Frequent customer involvement, and
- An emphasis on creating a demonstrable working product with each development cycle.

Some of the more popular approaches of Agile Methods are

- Rapid Application Development (RAD), eXtreme Programming (XP), and Scrum

Software Engineering Methodologies (Cont.)

RAD:

- Rapid software development methods are used primarily in data-intensive, business systems application development.
- The RAD method is enabled with special-purpose database development tools used by software engineers to quickly develop, test, and deploy new or modified business applications.

Software Engineering Methodologies (Cont.)

XP:

- This approach uses stories or scenarios for requirements, develops tests first, has direct customer involvement on the team (typically defining acceptance tests),
- uses pair programming, and provides for continuous code refactoring and integration.

- Stories are decomposed into tasks, prioritized, estimated, developed, and tested.
- Each increment of software is tested with automated and manual tests; an increment may be released frequently, such as every couple of weeks or so.

Software Engineering Methodologies (Cont.)

Scrum:

- This agile approach is more project management-friendly than the others.
- The scrum master manages the activities within the project increment; each increment is called a sprint and lasts no more than 30 days.

- A Product Backlog Item (PBI) list is developed from which tasks are identified, defined, prioritized, and estimated.
- A working version of the software is tested and released in each increment.
- Daily scrum meetings ensure work is managed to plan.